CS251 · Introduction to Software Engineering

S10 · May 2026

FinTrix

A Web-Based Budget Management System

Cairo University - Faculty of Computers and Artificial Intelligence

PROJECT TEAM

Omar Rady Tammam (20242227)

Asmaa Sameh Mohamed (20240084)

Adham Tamer Abdelmaougoud (20240067)

Mostafa Mohamed Ahmed (20240596)

CONTENTS

Table of Contents

01 Introduction & Motivation

02 Specifications & System Design

03 UML Diagrams & Architecture

04 Persistence & Implementation

05 Product Interface & Engineering

06 Extras & Collaboration

07 Conclusion & Future Vision

01 — Introduction

Project Overview: FinTrix is a web-based personal finance management system that empowers users to track, budget, and analyze their financial activities through a centralized, intuitive dashboard.

Track Income & Expenses

Record every transaction with categories, dates, and notes for full financial visibility

Monthly Budgets

Plan, allocate, and monitor monthly budgets across customizable spending categories

Financial Goals

Define savings targets, contribute regularly, and track progress towards each goal

Smart Notifications

Receive timely alerts when budgets approach thresholds or limits are exceeded

Reports & Analytics

Generate insightful financial reports with interactive pie and bar charts

Project Goals: Improve awareness, simplify tracking, and deliver actionable insights to users

02 — Motivation

Problem Statement: Traditional methods such as spreadsheets and manual tracking are inefficient, error-prone, and difficult to maintain at scale.

The Challenges

- **Tracking daily expenses:** Receipts get lost; cash purchases go unrecorded
- **Managing monthly budgets:** Manual budgeting fails to adapt to changing priorities
- **Understanding spending habits:** Without categorization, patterns remain invisible
- **Saving money effectively:** No structured way to set or track savings goals
- **Visualizing financial data:** Raw numbers don't communicate trends or insights

FinTrix Solutions

- **Centralized finance management:** A single dashboard for all financial activities
- **Automated budget monitoring:** Real-time tracking against category limits
- **Smart financial reporting:** Insights extracted automatically from your data
- **Goal tracking with contributions:** Define targets and watch progress accumulate
- **Interactive analytics:** Charts that turn raw data into clear stories

03 — Specifications

Functional Requirements: Six core modules define the system's behavior across authentication, transactions, budgets, analytics, goals, and notifications.

01. Authentication

- User registration
- User login & logout
- Validation & authentication
- Session management

02. Transaction Management

- Add income / expense
- Edit and delete transactions
- Filter and search
- Categorization

03. Budget Management

- Create monthly budgets
- Set per-category limits
- Monitor remaining budget
- Track utilization

04. Reports & Analytics

- Monthly financial reports
- Pie and bar charts
- Spending insights

05. Goals Management

- Create savings goals
- Track contribution progress
- Goal status updates

06. Notifications

- Budget threshold alerts
- Warning notifications
- Goal milestone updates

04 — Design

System Architecture: FinTrix follows a layered MVC architecture, separating presentation, business logic, and data access for maintainability and scalability.

Architecture Layers

Presentation Layer
HTML · CSS · JavaScript UI - Views & Templates

↕

Application Layer
Controllers & Services - Request Orchestration

↕

Business Logic Layer
Domain Rules & Calculations - Core Logic

↕

Data Access Layer
Repositories & ORM - Persistence Mapping

↕

Database Layer
SQLite — Persistent Storage - Storage Engine

Technology Stack

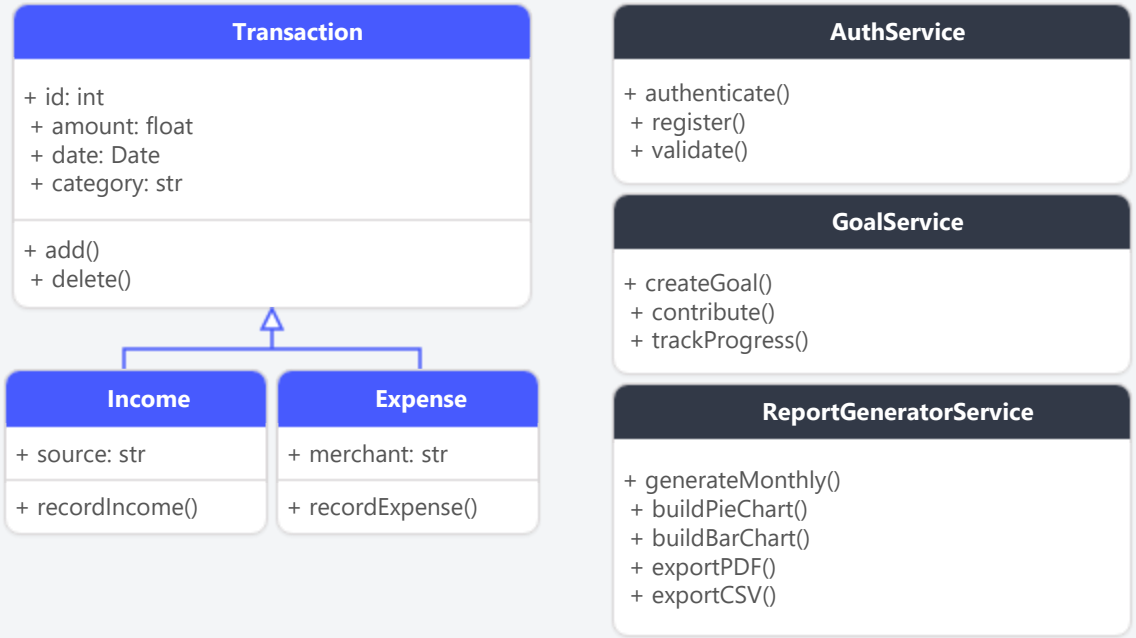
Frontend
HTML5, CSS3, JavaScript

Backend
Python 3, Django Framework, Service Layer

Database
SQLite, Django ORM

05 — UML Class Diagram

UML class diagram showing FinTrix's core domain entities, services, and their relationships across five modules.



Budget

+ id: int
+ month: str
+ totalLimit: float
+ categories: list

+ checkLimit()
+ getRemaining()

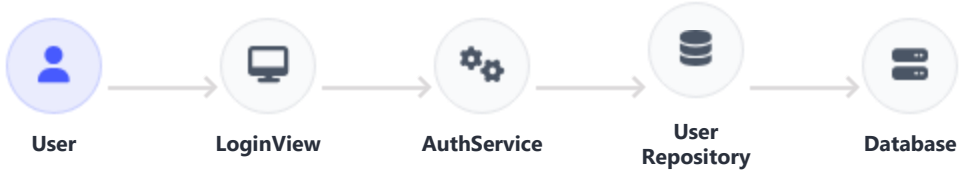
Key Relationships

- User owns Transactions (1:N)
- Transaction belongs to Category (N:1)
- Budget contains CategoryBudgets (1:N)

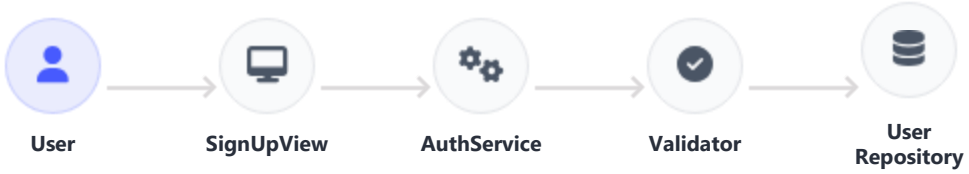
06 — UML Sequence Diagrams

Four key workflows captured as UML sequence diagrams: authentication, registration, transaction handling, and report generation.

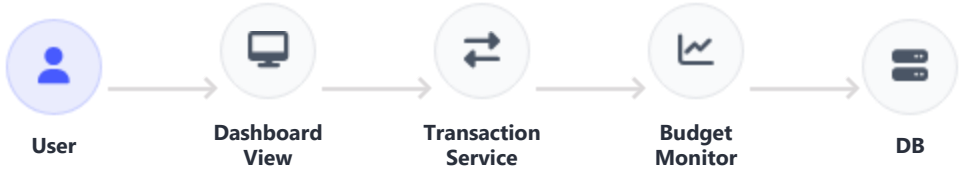
01. User Login Sequence



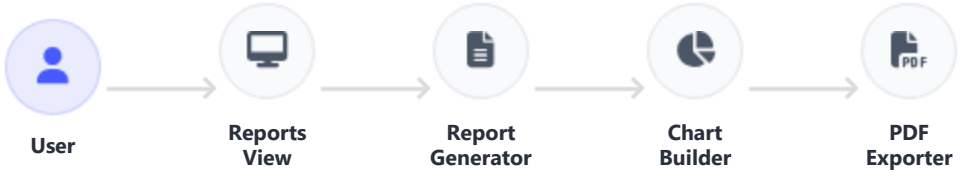
02. User Sign-Up Sequence



03. Add Transaction Sequence

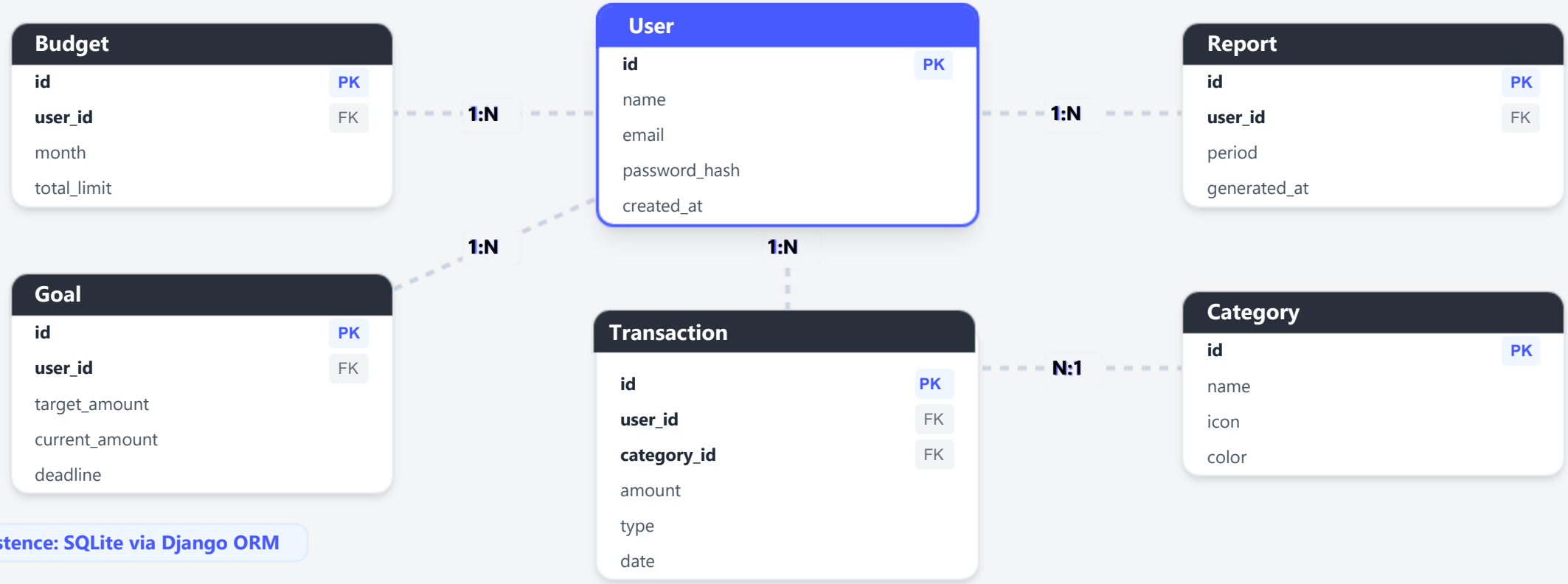


04. Generate Report Sequence



07 — Persistence

Relational schema implemented in SQLite, capturing users, transactions, budgets, categories, goals, and generated reports.



Persistence: SQLite via Django ORM

08 — Build

A modern web stack pairing Django on the backend with vanilla HTML, CSS, and JavaScript on the frontend, persisted via SQLite.

Layer 01 Frontend — HTML5 Semantic markup · CSS3 Responsive UI · JavaScript Interactions · Chart.js Visualizations

Layer 02 Backend — Python 3 Language · Django Framework · Service Layer Business logic · Repositories Data access

Layer 03 Database — SQLite Engine · Django ORM Mapping · Migrations Schema · Indexed FK lookups

Development Tools

VS Code	GitHub	PlantUML	Lucidchart	Visual Paradigm
Editor & debugging	Version control	UML diagrams	Architecture diagrams	Modeling & analysis

09 — Product

Six core screens cover the entire user journey — from authentication through daily use to financial reporting.

Login Page

Secure user authentication entry point

Sign-Up Page

New user registration with validation

Dashboard

Real-time financial overview & KPIs

Budget Page

Monthly limits across spending categories

Transactions Page

Searchable history of income and expenses

Reports Page

Charts, summaries & PDF/CSV export

10 — Engineering

FinTrix follows industry-standard engineering principles and patterns to ensure maintainable, extensible, and testable code.

SOLID Principles

S **Single Responsibility:** Each service handles one concern (Auth, Budget, Goals)

O **Open / Closed:** New report types extend the generator without modifying it

L **Liskov Substitution:** Income & Expense subtypes substitute Transaction freely

I **Interface Segregation:** Repositories expose only the methods their consumers need

D **Dependency Inversion:** Services depend on repository abstractions, not concrete DB

Design Patterns

- **Strategy:** Swappable report formats & chart strategies
- **Repository:** Abstracts data access from business logic
- **Factory Method:** Creates Income / Expense subtypes by type
- **MVC:** Separates models, views, and controllers
- **Service Layer:** Encapsulates business logic per module
- **Observer:** Triggers budget threshold notifications
- **Facade:** A simplified API hides ReportGenerator's subsystem complexity

11 — Extras & Collaboration

Bonus Features

Beyond core requirements, FinTrix delivers productivity, accessibility, and customization features inspired by modern fintech dashboards.

Export PDF Reports

Generate and download monthly financial reports as polished PDF documents

Export CSV Transactions

Export full transaction history to CSV for external analysis & backups

Dark Mode

Theme toggle for comfortable viewing in low-light environments

Search Transactions

Instant keyword search across descriptions, merchants, and categories

Filter by Date

Narrow transactions to specific date ranges with calendar pickers

Filter by Category

Drill into category-specific spending patterns and behavior

Filter by Type

Toggle between income, expense, or all transaction types instantly

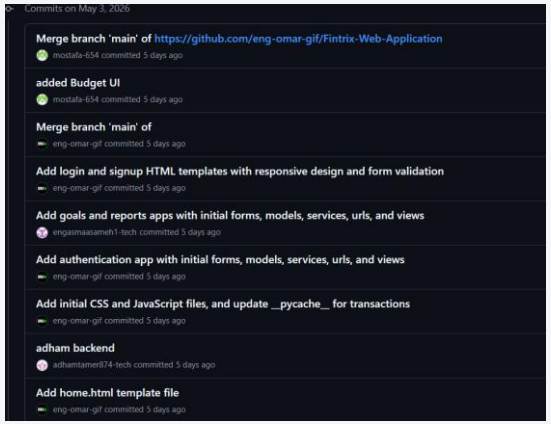
Responsive Design

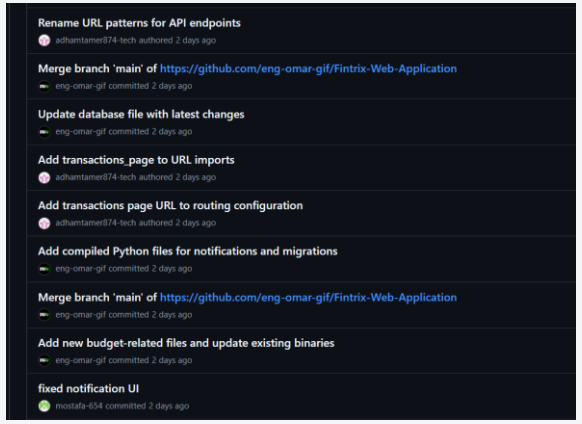
Optimized layouts adapt seamlessly across desktop, tablet, and mobile

GitHub Contributions

Task Distribution

- **Omar Rady Tammam:** Architecture · Authentication
- **Asmaa Sameh Mohamed:** Transactions & Budget
- **Adham Tamer Abdelmaougoud:** Reports & Analytics · Charts
- **Mostafa Mohamed Ahmed:** Goals · Notifications · UI





Repository: github.com/fintrix/app (Private Repository, Branch-based workflow · Code reviews)

Conclusion & Demo

What FinTrix Delivers

- Secure finance management
- Real-time budget monitoring
- Financial analytics & charts
- Goal tracking with progress
- Modern responsive UI
- PDF / CSV exports

Future Improvements

- Native mobile application
- AI-powered spending insights
- Cloud synchronization
- Multi-user collaboration
- Bank API integrations
- Recurring transactions engine

Thank You